

Projeto Final: Sistema de Gerenciamento de Estoque de uma Padaria

DESCRIÇÃO GERAL

Neste projeto final, vocês vão desenvolver um **Sistema Web de Controle de Estoque** voltado para uma padaria. O objetivo é construir uma aplicação funcional que resolva um problema básico do dia a dia do comércio: cadastrar, visualizar e gerenciar os ingredientes e produtos disponíveis.

O trabalho simula um ambiente profissional e o sistema será composto por duas partes que precisam se comunicar:

- **O Front-end (A Interface):** A tela onde o usuário interage, desenvolvida utilizando HTML, CSS e JavaScript puro. O sistema terá telas para o cadastro de novos produtos e um painel para visualizar e excluir itens.
- **O Back-end (O Servidor e Banco):** Onde as informações são processadas e salvas definitivamente.

PARTE 1: FRONT-END (Interface e Lógica Dinâmica)

Tecnologias: HTML, CSS e Vanilla JavaScript.

O Front-end do sistema será dividido em duas páginas distintas (**cadastro.html** e **estoque.html**), simulando um painel de controle real. Deve existir um menu de navegação simples (**links**) permitindo ao usuário transitar entre as duas telas.

1. Página de Cadastro (cadastro.html)

Esta página é dedicada exclusivamente à entrada de novos produtos ou ingredientes no sistema.

- **O Formulário:** Deve conter uma variedade de campos para registrar o item de forma completa:
 - Nome do Item
 - Categoria (Caixa de seleção - `<select>` com opções como: Pães, Doces, Frios)
 - Descrição (Área de texto - `<textarea>` para detalhes como "Farinha especial para fermentação natural")
 - Data de Validade
 - Quantidade
 - Preço Unitário
 - Um botão de envio
- **Ação e Feedback:** Ao clicar em cadastrar, o JavaScript deve validar se os campos obrigatórios não estão vazios. Após enviar os dados com sucesso, o formulário deve ser limpo e um aviso de "Cadastrado com sucesso!" deve aparecer na tela.

2. Página de Gestão de Estoque (estoque.html)

Página onde o usuário visualiza e gerencia o que já existe na padaria.

- **Produtos:** Uma **tabela ou lista de cards** gerada dinamicamente via JavaScript assim que a página carregar, exibindo todos os itens.
- **Ação de Exclusão (Deletar):** Cada linha da **tabela ou lista** deve conter um botão de exclusão (Lixeira ou "X"). Ao clicar nele, o JS deve enviar o comando de exclusão para o servidor e, em caso de sucesso, remover a respectiva linha da tabela do HTML.
- **Ação extra (Editar):** Adicione um botão [Editar] ao lado do botão de deletar na tabela ou lista.
 - **O Modal:** Crie uma janela flutuante (Modal) no HTML da página de estoque, que comece escondida no CSS (display: none). Dentro dela, coloque um formulário idêntico ao de cadastro.
 - **A Lógica:** Ao clicar em [Editar], o JS captura os dados daquela linha, preenche o formulário do Modal automaticamente e faz a janela aparecer na tela.
 - **Ao clicar em "Salvar Alterações",** o JS empacota os dados e envia um fetch usando o método PUT para a rota criada no backend responsável por editar um produto.
 - Se o servidor retornar sucesso, feche o Modal e recarregue a lista de produtos

PARTE 2: BACK-END (Servidor e Banco de Dados)

Tecnologia: Node.js com Express e MongoDB (Mongoose).

Nesta etapa, vocês construirão a API RESTful que servirá como o "motor" do sistema, além de configurar o banco de dados não relacional para salvar as informações da padaria de forma permanente.

1. O Banco de Dados (MongoDB)

Vocês devem configurar a conexão com o MongoDB (podem utilizar o MongoDB Atlas para hospedar na nuvem gratuitamente) e criar o "Schema" (modelo) para a coleção de produtos. O modelo deve refletir exatamente os dados enviados pelo Front-end:

- nome: Texto (String), obrigatório.
- categoria: Texto (String), obrigatório.
- descricao: Texto (String).
- data_validade: Texto ou Data (String/Date).
- quantidade: Número (Number), obrigatório e não pode ser negativo.
- preco: Número (Number), obrigatório.

2. A Construção da API (Rotas e Endpoints)

A API deve ser o canal de comunicação exclusivo entre a interface e o banco de dados.

Vocês precisarão criar as seguintes rotas para atender aos requisitos do Front-end:

- **Criar Produto (POST /api/produtos):** Esta rota deve receber o corpo da requisição (body) contendo os dados do formulário de cadastro. O servidor deve validar se os campos obrigatórios estão presentes e, em seguida, salvar o novo documento no MongoDB. Deve retornar o status 201 (Created) em caso de sucesso.
- **Listar Estoque (GET /api/produtos):** Rota chamada assim que a página de estoque é carregada. O servidor deve buscar todos os documentos salvos na coleção do MongoDB e retorná-los em formato JSON (um Array de Objetos).
- **Deletar Produto (DELETE /api/produtos/:id):** Esta rota será acionada pelo botão de lixeira. Ela deve capturar o ID do produto passado na URL, buscar esse documento específico no MongoDB e removê-lo. Deve retornar um status 200 (OK) para avisar o Front-end que a exclusão foi concluída.
- **Editar Produto (PUT ou PATCH /api/produtos/:id):** Acionada pelo formulário do Modal de edição. A rota recebe o ID do produto pela URL e os novos dados pelo corpo da requisição (body). O servidor deve encontrar o produto no MongoDB, atualizar os valores modificados e salvar a alteração.